

Program-2: Develop a Multi-Stage Dockerfile for Container Orchestration

Step 1: Create Project Directory

```
mkdir Program2  
cd Program2
```

Step 2: Create Dockerfile (Multi-Stage)

```
nano Dockerfile
```

Add the following content:

```
### Start the Multi-Stage Dockerfile (Build & Production)
```

```
# ----- Stage 1: Build Stage -----
```

```
FROM node:20-alpine AS builder
```

```
# Set working directory
```

```
WORKDIR /app
```

```
# Copy package files and install dependencies
```

```
COPY package.json package-lock.json ./
```

```
RUN npm install
```

```
# Copy application source code
```

```
COPY . .
```

```
# Build the application
```

```
RUN npm run build
```

```
# ----- Stage 2: Production Stage -----
```

```
FROM node:20-alpine
```

```
# Set working directory
```

```
WORKDIR /app
```

```
# Copy only required files from build stage
COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
```

```
# Expose application port
EXPOSE 3000
```

```
# Start the application
CMD ["node", "dist/index.js"]
```



```
ganeshv@ganeshv-HP-ENVY-x360-m6-Convertible: ~/Desktop/devopslab/secondprogram
GNU nano 7.2 Dockerfile
#Stage 1: Building Stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json ./
RUN npm install

COPY src ./src
RUN mkdir -p dist && cp src/index.js dist/
RUN npm run build

#Stage 2: Production
FROM node:20-alpine
WORKDIR /app

COPY --from=builder /app/package.json ./
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist

EXPOSE 3000

CMD ["npm", "start"]

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line M-E Redo
```

Save and exit.

Step 3: Create Source Folder and index.js

```
mkdir src
nano src/index.js
```

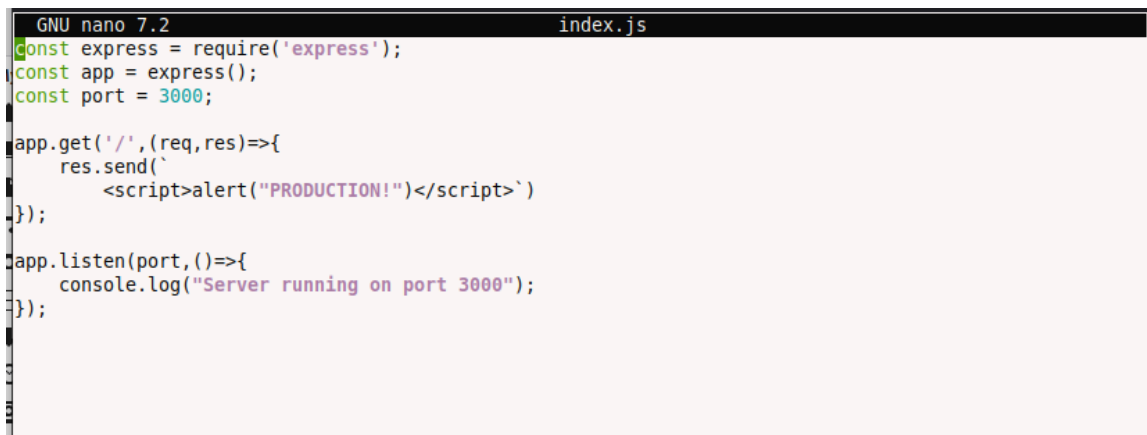
Add:

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from multi-stage Docker!');
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Save and exit.



```
GNU nano 7.2 index.js
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send(`
    <script>alert("PRODUCTION!")</script>`
  );
});

app.listen(port, () => {
  console.log("Server running on port 3000");
});
```

Step 4: Create package.json

Run:

```
npm init -y
```

Step 5: Install Node Modules

```
npm install
```

Step 6: Build Docker Image

docker build -t program2 .

```
ganeshv@ganeshv-HP-ENVY-x360-m6-Convertible: ~/Desktop/devopslab/secondprogram
ganeshv@ganeshv-HP-ENVY-x360-m6-Convertible:~/Desktop/devopslab/secondprogram$ sudo docker build -t secondprogram .
[+] Building 22.6s (15/15) FINISHED
=> [internal] load build definition from Dockerfile                                0.05s
=> => transferring dockerfile: 444B                                              0.05s
=> [internal] load metadata for docker.io/library/node:20-alpine                 3.55s
=> [internal] load .dockerignore                                                  0.05s
=> => transferring context: 2B                                                    0.05s
=> [builder 1/7] FROM docker.io/library/node:20-alpine@sha256:658d0f63e501824d6c23e06d4bb95c71e7d704537c9d92 10.35s
=> => resolve docker.io/library/node:20-alpine@sha256:658d0f63e501824d6c23e06d4bb95c71e7d704537c9d9272f488ac0 0.05s
=> => sha256:fcbb8f7d018707c656a4da2eea8a15f2893d2258093fea9ff2ea5ealcb82112 1.72kB / 1.72kB 0.05s
=> => sha256:e80b0510ba947015cacdddea3d23dcdc761e399971c074a1bf32eea7e44510524 6.52kB / 6.52kB 0.05s
=> => sha256:1074353eecd0db2c1d81d5af2671e56e0cf5738486f5762609ea33d606f88612 3.86MB / 3.86MB 2.05s
=> => sha256:8d06ba6946d1f299d8f962e37906c77006df33d47050430a57f7893ec35af697 42.78MB / 42.78MB 8.25s
=> => sha256:cb3325e64457574e24f92246e3da3376946e473d636209e1390eac47b50b26a3 1.26MB / 1.26MB 1.35s
=> => sha256:658d0f63e501824d6c23e06d4bb95c71e7d704537c9d9272f488ac03a370d448 7.67kB / 7.67kB 0.05s
=> => sha256:fd1849a5c548bc65ee47a64498951bda5d40e87f008efe9dca69b5c8cdec7a52 443B / 443B 2.25s
=> => extracting sha256:1074353eecd0db2c1d81d5af2671e56e0cf5738486f5762609ea33d606f88612 0.35s
=> => extracting sha256:8d06ba6946d1f299d8f962e37906c77006df33d47050430a57f7893ec35af697 1.75s
=> => extracting sha256:cb3325e64457574e24f92246e3da3376946e473d636209e1390eac47b50b26a3 0.15s
=> => extracting sha256:fd1849a5c548bc65ee47a64498951bda5d40e87f008efe9dca69b5c8cdec7a52 0.05s
=> [internal] load build context                                                  0.05s
=> => transferring context: 346B                                                  0.05s
=> [builder 2/7] WORKDIR /app                                                    0.55s
=> [builder 3/7] COPY package.json ./                                           0.05s
=> [builder 4/7] RUN npm install                                                 6.85s
=> [builder 5/7] COPY src ./src                                                  0.05s
=> [builder 6/7] RUN mkdir -p dist && cp src/index.js dist/                    0.25s
=> [builder 7/7] RUN npm run build                                              0.55s
=> [stage-1 3/5] COPY --from=builder /app/package.json ./                      0.05s
=> [stage-1 4/5] COPY --from=builder /app/node_modules ./node_modules          0.25s
=> [stage-1 5/5] COPY --from=builder /app/dist ./dist                          0.05s
=> exporting to image                                                            0.25s
=> => exporting layers                                                            0.25s
=> => writing image sha256:362c3fb71ccd731f09e3664168be13ec960fdb4e3a1901332dbb792b966bc800 0.05s
=> => naming to docker.io/library/secondprogram                                0.05s
ganeshv@ganeshv-HP-ENVY-x360-m6-Convertible:~/Desktop/devopslab/secondprogram$
```

Verify:

docker images

Step 7: Run Docker Container

docker run -it -p 3000:3000 program2

```
ganeshv@ganeshv-HP-ENVY-x360-m6-Convertible:~/Desktop/devopslab/secondprogram$ sudo docker run -p 3000:3000 secondprogram
> second@1.0.0 start
> node dist/index.js

Server running on port 3000
```

Step 8: Test the Application

Open browser:

http://localhost:3000

Output:

Hello from multi-stage Docker!

